

Cognitive Work Analysis Software Tool Project Plan

The Phantom Thieves

Ash Franklin

Ata Noor

Cole DenBleyker

Ella Natter

Jensen DeRosier

Zach Brown

Cognitive Work Analysis Software Tool Project Plan.....	0
The Phantom Thieves.....	0
Project Goals and Scope.....	3
1. Project Goals.....	3
2. Project Scope.....	4
2.1 In Scope.....	4
2.2 Out of Scope.....	4
2.3 Assumptions and Constraints.....	4
Milestones & Major Deliverables.....	5
Milestone 1: Core System Infrastructure and Project Management.....	5
Milestone 2: Traceability, Provenance, and Iteration Safeguards.....	6
Milestone 3: Stage 1 - Work Domain Analysis (WDA).....	6
Milestone 4: Stage 2 - Work Organization Analysis (WOA).....	7
Milestone 5: Stage 3 - Cognitive Transformations Analysis (CTA).....	7
Milestone 6: Stage 4 - Strategies Analysis (STR).....	8
Milestone 7: Stage 5 - Cognitive Processing Analysis (CPA).....	8
Milestone 8: Stage 6 - Social Transactions Analysis (STA).....	9
Milestone 9: Usability, Guidance, and Safeguards.....	9
Milestone 10: Open-Source Release and Documentation.....	10
Software Requirements Specification.....	11
1. System Context and Purpose.....	11
2. Functional Requirements.....	11
2.1 General System Requirements (GEN).....	11
2.2 Stage 1 - Work Domain Analysis (WDA).....	13
2.3 Stage 2 - Work Organization Analysis (WOA).....	16
2.4 Stage 3 - Cognitive Transformations Analysis (CTA).....	18
2.5 Stage 4 - Strategies Analysis (STR).....	19
2.6 Stage 5 - Cognitive Processing Analysis (CPA).....	20
2.7 Stage 6 - Social Transactions Analysis (STA).....	21
3. Data Management Requirements (DATA).....	21
4. Usability and Accessibility Requirements.....	22
5. Security and Privacy Requirements (SEC).....	23
6. Open Source and Licensing Requirements (OSS).....	24
7. Deployment Requirements (DEP).....	24
8. Interpretive Notes and Design Assumptions.....	24
User Stories.....	26
A. Core Project & Data Management Stories.....	26
B. Stage-Specific User Stories.....	27

Stage 1 - Work Domain Analysis (WDA).....	27
Stage 2 - Work Organization Analysis (WOA).....	28
Stage 3 - Cognitive Transformations Analysis (CTA).....	28
Stage 4 - Strategies Analysis (STR).....	29
Stage 5 - Cognitive Processing Analysis (CPA).....	29
Stage 6 - Social Transactions Analysis (STA).....	29
C. Safeguards & Meta Stories.....	30
Work Breakdown Structure.....	31
Communication and Stakeholder Management Plan.....	36
Risk Management Plan.....	38
1. Technical Risks.....	38
2. Architectural Risks.....	39
3. Process and Schedule Risks.....	40
4. Quality & Maintenance Risks.....	41
Development Methodology.....	43
Planned Activities and Artifacts.....	43
Roles.....	43
Standards and Quality Practices.....	44
Tools.....	44
Schedule.....	44
Process & Project Metrics.....	45
Process.....	45
Metrics.....	45
Tools & Techniques.....	46
Summary.....	46
Project Management Tools.....	46
Technical Development Tools.....	46
Technical Implementation Tools.....	47
Domain Model.....	48

Project Goals and Scope

1. Project Goals

The goal of this project is to design and implement the Cognitive Work Analysis Software Tool (CWA-ST), an open-source software tool that supports the complete execution of Cognitive Software Analysis (CWA) across all six stages, as defined in established literature. The system is intended to reduce the time, effort, and cognitive burden associated with conducting comprehensive CWAs while preserving human analytical authority and methodological rigor. Specifically, the project aims to:

1. **Support End-to-End CWA:** Provide integrated support for all six stages of CWA - Work Domain Analysis, Work Organization Analysis, Cognitive Transformations Analysis, Strategies Analysis, Cognitive Processing Analysis, and Social Transactions Analysis - all within a coherent workflow.
2. **Preserve Human Control and Authority:** ensure that all automated or assisted system outputs function strictly as decision-support artifacts, not as authoritative analyses. Analysts retain full control over acceptance, modification, rejection, and interpretation of all artifacts
3. **Improve Efficiency and Repeatability:** Streamline document analysis, qualitative data organization, and artifact construction in order to make CWA more practical for large-scale or repeated use, while supporting greater consistency across analysts given similar inputs.
4. **Ensure Traceability, Provenance, and Verifiability:** Maintain explicit traceability between all analytical artifacts and their source data, along with provenance metadata that documents how each artifact was created and modified. This supports auditability, review, and future research into the reliability and validity of CWA.
5. **Remain Faithful to the CWA Framework:** Implement functionality that reflects the current state of the CWA framework as reported in open literature, without introducing methodological critiques, reinterpretations, or enforced structures.
6. **Enable Future Evaluation and Extension:** Produce outputs that can be used not only for system and interface design, but also for future evaluation of the CWA framework itself, including studies of analyst variability, usability, and methodological effectiveness.

2. Project Scope

2.1 In Scope

The project includes the design and implementation of a software system that:

1. Supports all six stages of CWA in a single environment, including the creation, editing, validation, and export of standard CWA artifacts.
2. Facilitate the automation of creating standard CWA artifacts in a variety of formats.
3. Handles qualitative, text-based data such as documents, interview notes, and survey responses as primary inputs .
4. Provides assisted analysis features (e.g., candidate identification, templates, validation checks) that are explainable and fully traceable.
5. Stores all data in a structured, persistent format that preserves stage relationships, traceability, and provenance.
6. Provides guidance and examples to support analysts with limited prior exposure to CWA.
7. Is released as open-source software, with complete user and developer documentation and example datasets free of sensitive data.
8. Is accessible via a web-based interface and available for free public use, with access control for proprietary or sensitive projects.

2.2 Out of Scope

The project explicitly does not include:

1. Critique, modification, or extension of the CWA framework itself.
2. Enforcement of a single canonical structure for CWA artifacts (e.g., mandatory diagrams)
3. Multi-analyst simultaneous collaboration, role-based workflows, or real-time coediting.
4. Domain-specific customization or optimization beyond what is normally achieved.
5. Collection or mandatory storage of identifiable human subject data.
6. Large scale validation studies, or inter-analyst reliability experiments, though the tool is designed to enable such studies in the future.

2.3 Assumptions and Constraints

1. The system assumes a single primary analyst per project, with one source of human input at a time
2. All automation mechanisms must be tractable, explainable, and verifiable, and must not function as opaque “black-box” analyses
3. The system must remain faithful to published CWA literature

4. Security and privacy mechanisms are provided to support compliance with applicable Institutional Review Board (IRB) requirements, but responsibility for appropriate use remains with the analyst.

Milestones & Major Deliverables

The development of CWAIST will follow an Agile Scrum methodology, delivering a usable and testable Increment at the end of each sprint. Milestones represent logical groupings of functionality that may span multiple sprints but cumulate in clearly defined deliverables with verifiable acceptance criteria. All milestone acceptance criteria must be satisfied before the milestone is considered complete.

It is important to note that this section is about *methodological staging*, and will not be the actual order in which software development features are developed. These milestones represent logical clusters of capabilities, not chronological steps.

Milestone 1: Core System Infrastructure and Project Management

Objective: Establish the foundational infrastructure required to support all subsequent CWA functionality and iterative development.

Major Deliverables:

1. Web-based application framework
2. Project creation and persistence mechanisms
3. Structured artifact storage
4. Optional user account creation and authentication
5. Initial deployment configuration

Acceptance Criteria:

1. Analysts can create, open, rename, and delete multiple CWA projects (GEN-01).
2. Each project persists independently and reloads without data loss after system restart (GEN-01, GEN-02).
3. Artifacts are stored in a structured database that enforces stage relationships (GEN-02).
4. User accounts can be created, logged into, logged out of, and deleted (GEN-09).
5. Authentication remains optional and does not block access to creation of projects made without persistent storage (GEN-09).
6. The system is accessible through a web browser without a paywall (DEP-01, DEP-02).

Milestone 2: Traceability, Provenance, and Iteration Safeguards

Objective: ensure methodological integrity through traceability, provenance tracking, and controlled iteration

Major Deliverables:

1. Artifact-to-source traceability system
2. Provenance metadata framework
3. Change impact detection and reconciliation workflow
4. Draft labeling for automated outputs

Acceptance Criteria:

1. Every artifact includes references to its originating source data (GEN-03).
2. Analysts can navigate from any artifact directly to its source documents (GEN-03).
3. Removal of a source document triggers warnings for dependent artifacts (GEN-03).
4. Each artifact records creator, creation method, timestamps, and modification history (GEN-04).
5. Automatically generated artifacts are clearly labeled as drafts (GEN-07).
6. Editing upstream artifacts will cause downstream elements to lose their progress. The user will be made well aware of this through warnings (GEN-05, GEN-06).
7. Analysts can accept, reject, or manually reconcile propagated changes, and all decisions are logged (GEN-05).

Milestone 3: Stage 1 - Work Domain Analysis (WDA)

Objective: Provide full support for constructing, validating, and exporting the Abstraction–Decomposition Space.

Major Deliverables:

1. Source document upload and viewing
2. Assisted and manual creation of WDA artifacts
3. Abstraction Hierarchy and ADS creation
4. Validation and export capabilities

Acceptance Criteria:

1. Supported document formats upload successfully and are viewable post-upload (WDA-01, DATA-01, DATA-02).
2. The system suggests candidate WDA elements with navigable source references and deterministic outputs (WDA-02, WDA-03, WDA-05, WDA-07, WDA-09, WDA-11).

3. No candidate artifact is committed without analyst confirmation (WDA-02, WDA-07, WDA-09, WDA-11).
4. Analysts can manually create, edit, and delete all WDA artifacts and links (WDA-04 through WDA-16).
5. The system generates a five-level Abstraction Hierarchy reflecting committed artifacts (WDA-13).
6. Validation identifies orphaned nodes or missing links as warnings only (WDA-17).
7. WDA artifacts export correctly (WDA-18).

Milestone 4: Stage 2 - Work Organization Analysis (WOA)

Objective: Enable modeling of work situations, tasks, and contextual activity while preserving iterative flexibility.

Major Deliverables:

1. Work Situation and Task Management
2. Contextual Activity Matrix Editor
3. Span-of-action definition and validation

Acceptance Criteria:

1. The system assists with identifying Work Situations and Tasks while leaving final control to the analyst (WOA-01 through WOA-04).
2. Analysts can associate Work Tasks with Domain or Physical Functions or leave them unlinked (WOA-05, WOA-06).
3. Analysts can not revise previous stages unless they wish to undo work on the current stage (WOA-07).
4. A Contextual Activity Matrix is generated and fully editable (WOA-08, WOA-09).
5. Desired and Potential Spans of Action are definable and validated with non-blocking warnings (WOA-10, WOA-11).
6. The Contextual Activity Matrix exports correctly (WOA-12).

Milestone 5: Stage 3 - Cognitive Transformations Analysis (CTA)

Objective: Support flexible modeling of cognitive states, processes, and decision structures.

Major Deliverables:

1. State–Process Diagram editor
2. Editable Decision Ladder templates
3. Validation and export functionality

Acceptance Criteria:

1. Analysts can create zero or more State–Process Diagrams per Work Task (CTA-01).
2. Cognitive States and Processes are fully editable without enforced ordering (CTA-02, CTA-03).
3. Decision Ladder templates are attempted to be automatically generated from previous stages but remain fully editable and removable (CTA-05, CTA-06).
4. Validation identifies disconnected or orphaned elements without enforcing canonical structures (CTA-04, CTA-07).
5. CTA artifacts export correctly in supported formats (CTA-08).

Milestone 6: Stage 4 - Strategies Analysis (STR)

Objective: Capture variability in cognitive strategies and underlying reasoning.

Major Deliverables:

- a. Strategy identification and association tools (automations)
- b. Strategy tables createable
- c. Export capabilities

Acceptance Criteria:

- a. Analysts can associate multiple Cognitive Strategies with Decision Ladder processes (STR-01, STR-02).
- b. Reasoning, conditions, and heuristics are fully editable and support multiple rationales (STR-03).
- c. Strategy tables are generated accurately and remain editable (STR-04, STR-05).
- d. Strategy artifacts export correctly (STR-06).

Milestone 7: Stage 5 - Cognitive Processing Analysis (CPA)

Objective: Document cognitive processing modes and information processing demands.

Major Deliverables:

- a. Processing mode classification tools
- b. Cognitive Processing tables
- c. Export functionality

Acceptance Criteria:

- a. Analysts can classify strategies as skill-, rule-, or knowledge-based without automatic enforcement (CPA-01).

- b. Information use, transformation, and procedures are editable and persist across sessions (CPA-02, CPA-03).
- c. Cognitive Processing tables accurately map tasks to processing modes (CPA-04).
- d. CPA artifacts export correctly (CPA-05).

Milestone 8: Stage 6 - Social Transactions Analysis (STA)

Objective: model social, organizations, and transactional relationships

Major Deliverables:

- a. Social Transactions Docket
- b. Transaction Network Diagram Editor
- c. Validation and export features

Acceptance Criteria

- a. A Social Transactions Docket is generated from existing Work Tasks (STA-01).
- b. Analysts can assign agents, transaction demands, and transaction modes (STA-02, STA-03).
- c. Transaction Network Diagrams are auto-generated and editable except for Work Tasks (STA-04, STA-05).
- d. Validation identifies orphaned elements and missing tasks (STA-06).
- e. STA artifacts export correctly (STA-07).

Milestone 9: Usability, Guidance, and Safeguards

Objective: ensure accessibility for novice users while preserving expert flexibility

Major Deliverables

- a. Context-sensitive guidance and examples
- b. Non-linear navigation across stages
- c. Final validation

Acceptance Criteria:

- a. Analysts with minimal prior exposure can complete a basic CWA workflow (USE-01).
- b. Instructional content and examples are available at each stage without blocking workflow (USE-02).
- c. Analysts can navigate non-linearly across all CWA stages without data loss (USE-03).
- d. All validation remains advisory and non-blocking (C1 user stories).

Milestone 10: Open-Source Release and Documentation

Objective: Prepare CWAIST for independent use and public contribution.

Major Deliverables:

- a. Public source code repository
- b. User and developer documentation
- c. Example datasets and projects

Acceptance Criteria:

- a. Source code is publicly accessible in a version-controlled repository (OSS-03).
- b. User and developer documentation enables independent installation and execution (OSS-02).
- c. Example datasets and mock projects contain no sensitive data (OSS-02).
- d. CI pipelines enforce linting, testing, container builds, and coverage thresholds as specified in quality practices.

Software Requirements Specification

1. System Context and Purpose

CWAST is a software tool intended to support analysts in conducting complete Cognitive Work Analysis (CWA) across all six stages defined in the published literature. The system is designed to reduce the time and effort required for document analysis, data organization, and artifact construction, all while maintaining human control. Automatically generated outputs are decision-support artifacts, not authoritative analyses. All outputs shall remain editable and verifiable by human analysts. Interpretive assumptions and design principles are documented in Section 8.

2. Functional Requirements

2.1 General System Requirements (GEN)

GEN-01: The system shall allow analysts to create, open, save, and manage multiple CWA projects.

Acceptance Criteria:

1. Users can create, rename, open, and delete projects
2. Each project persists independently
3. Projects reload without data loss

GEN-02: The system shall store all CWA artifacts in a structured, persistent database.

Acceptance Criteria:

1. All artifacts persist across sessions
2. Data schema enforces stage ordering and artifact dependencies
3. System recovers state after restart

GEN-03: The system shall maintain traceability between all CWA artifacts and their source data (documents, surveys, interviews).

Acceptance Criteria:

1. Every artifact node includes references to its originating source file
2. Analysts can navigate from any artifact to its source data.
3. Removal of a source document triggers a warning for dependent artifacts.
4. Traceability information is preserved in all exported formats.

GEN-04: The system shall record provenance metadata for each artifact, including creation method (manual or assisted), timestamp, and analyst.

Acceptance Criteria:

1. Each artifact records: creator (user/system), creation timestamp, and modification history.
2. Provenance metadata is viewable by the domain expert.
3. Provenance metadata survives export and re-import.
4. Automatically generated artifacts are clearly labeled as such in provenance records.

GEN-05: The system shall enforce stage completion confirmation and freeze artifacts upon advancement to the next stage.

Acceptance Criteria:

1. Domain experts must explicitly confirm completion before advancing stages
2. Once confirmed, artifacts from completed states will become read-only
3. The system clearly indicates which stages are frozen

GEN-06: The system shall require deletion of downstream artifacts when a domain expert elects to revise artifacts from a completed earlier stage.

Acceptance Criteria:

1. The system warns the user which downstream stages will be discarded
2. Domain experts must explicitly confirm deleting before reverting
3. All artifacts from reverted downstream stages are permanently removed
4. The system returns the user to the last editable stage

GEN-07: The system shall treat all automatically generated content as editable drafts.

Acceptance Criteria:

1. All automatically generated artifacts are visually and programmatically distinguishable from analyst-confirmed artifacts.
2. Domain experts can modify, delete, or replace any automatically generated artifact without system restriction
3. No downstream stage requires acceptance of an automatically generated artifact to proceed.
4. Exported artifacts do not mark automatically generated content as authoritative unless explicitly confirmed by the analyst.

GEN-08: The system shall provide context-sensitive guidance and examples at each stage of CWA.

Acceptance Criteria:

1. Each stage includes contextual help on how to use the tool
2. Examples are accessible from UI
3. Help does not block workflow

GEN-09: The system shall allow the creation and persistence of individual users (This criteria is an assumption based on the existence of [GEN-01])

Acceptance Criteria:

1. User creation is possible for a new user
2. Valid and invalid logins are handled correctly
3. Authentication remains optional
4. Project can be securely shared with controlled access

GEN-10: The system shall allow no-account users to create projects although they will not be able to store their work on the website without an account (Assumed based off [DEP-02])

Acceptance Criteria:

1. Creation of projects is available without an account
2. Artifacts created during the session are not persisted until an account is created

2.2 Stage 1 - Work Domain Analysis (WDA)

WDA-01: The system shall allow analysts to upload and store source documents relevant to the work domain.

Acceptance Criteria:

1. More than five documents are supported per project
2. Supported formats upload successfully
3. Documents are viewable post-upload

WDA-02: The system shall automatically identify candidate Domain Purposes from uploaded documents in a deterministic manner.

Acceptance Criteria:

1. For each candidate element, the system displays at least one source reference.
2. The system does not auto-commit candidates into the Abstraction Hierarchy without user confirmation.
3. Given the same input documents and configuration, the system produces the same candidate outputs.

WDA-03: The system shall present source references for each candidate Domain Purpose.

Acceptance Criteria:

1. Each candidate lists at least one source
2. Sources are navigable to the originating document

WDA-04: The system shall allow analysts to create, edit, or delete Domain Purposes manually.

Acceptance Criteria:

1. Domain experts can create, edit, or delete entries

2. Changes require reconfirmation before advancing stages

WDA-05: The system shall automatically identify candidate Domain Values.

Acceptance Criteria:

1. Candidates include source references
2. The system does not auto-commit candidates into the Abstraction Hierarchy without analyst confirmation.
3. Manual override always possible

WDA-06: The system shall allow analysts to create, edit, or delete Domain Values.

Acceptance Criteria:

1. Candidates generated with sources
2. Manual override always possible

WDA-07: The system shall automatically identify candidate Domain Functions.

Acceptance Criteria:

1. For each candidate element, the system displays at least one source reference.
2. The system does not auto-commit candidates into the Abstraction Hierarchy without domain expert confirmation.
3. Given the same input documents and configuration, the system produces the same candidate outputs.

WDA-08: The system shall allow analysts to create, edit, or delete Domain Functions.

Acceptance Criteria:

1. Candidates are suggestions only
2. Analyst can freely modify

WDA-09: The system shall automatically identify candidate Physical Functions.

Acceptance Criteria:

1. For each candidate element, the system displays at least one source reference.
2. The system does not auto-commit candidates into the Abstraction Hierarchy without domain expert confirmation.
3. Given the same input documents and configuration, the system produces the same candidate outputs.
4. Deterministic output is enforced

WDA-10: The system shall allow analysts to create, edit, or delete Physical Functions.

Acceptance Criteria:

1. Candidates are suggestions only
2. Analyst can freely modify

WDA-11: The system shall automatically identify candidate Physical Objects.

Acceptance Criteria:

1. For each candidate element, the system displays at least one source reference.
2. The system does not auto-commit candidates into the Abstraction Hierarchy without domain expert confirmation.
3. Given the same input documents and configuration, the system produces the same candidate outputs.
4. Automation is deterministic

WDA-12: The system shall allow domain experts to create, edit, or delete Physical Objects.

Acceptance Criteria:

1. Objects link to sources
2. Analyst can add non-documented objects

WDA-13: The system shall generate an Abstraction Hierarchy containing all five standard levels.

Acceptance Criteria:

1. Five standard levels are created
2. Hierarchy reflects committed artifacts

WDA-14: The system shall allow domain experts to edit any node within the Abstraction Hierarchy.

Acceptance Criteria:

1. Nodes can be edited or deleted
2. Links update accordingly

WDA-15: The system shall allow domain experts to define additional decomposition levels beyond system, unit, component, and part.

Acceptance Criteria:

1. Analyst can add or remove levels
2. Visualization updates correctly

WDA-16: The system shall allow domain experts to create, edit, or delete means–ends links.

Acceptance Criteria:

1. Domain experts can add, edit, or delete links
2. Links persist across sessions

WDA-17: The system shall validate the Abstraction–Decomposition Space by identifying orphaned nodes or missing links.

Acceptance Criteria:

1. The system identifies nodes missing means–ends links.
2. The system identifies discontinuities across abstraction levels.
3. Validation results are presented as warnings, not blockers.

WDA-18: The system shall allow export of the Abstraction Hierarchy and Abstraction–Decomposition Space.

Acceptance Criteria:

1. Exports reflect current state
2. Formats render correctly
3. Document is in a spreadsheet format

2.3 Stage 2 - Work Organization Analysis (WOA)

WOA-01: The system shall automatically identify candidate Work Situations from documents and interviews.

Acceptance Criteria:

1. Candidates may be suggested
2. Each candidate includes source references
3. Analyst control the final confirmed list

WOA-02: The system shall allow domain experts to create, edit, or delete Work Situations.

Acceptance Criteria:

4. Candidates may be suggested
5. Analyst controls final list

WOA-03: The system shall automatically identify candidate Work Tasks associated with Work Stations.

Acceptance Criteria:

1. Tasks include text descriptions
2. Each task includes source references where available
3. Automation is deterministic
4. Domain expert can fully edit or replace tasks

WOA-04: The system shall allow domain experts to create, edit, or delete Work Tasks and their descriptions.

Acceptance Criteria:

5. Tasks include descriptions
6. Tasks are fully editable

WOA-05: The system shall allow domain experts to associate Work Tasks with Domain or Physical Functions.

Acceptance Criteria:

1. Tasks can be mapped to one or more functions
2. Multiple mappings allowed

WOA-06: The system shall allow Work Tasks to exist independently of the Abstraction–Decomposition Space.

Acceptance Criteria:

1. Tasks can exist without ADS linkage

WOA-07: The system shall enforce stage completion confirmation and downstream discard when revising prior stages.

Acceptance Criteria:

1. Stage 1 artifacts are read-only after WOA confirmation
2. Revising Stage 1 requires discarding all WOA artifacts
3. The system clearly identifies which data will be removed
4. Analysts must explicitly confirm before reverting

WOA-08: The system shall generate a Contextual Activity Matrix based on confirmed Work Situations and Tasks.

Acceptance Criteria:

1. Matrix reflects tasks and situations
2. Matrix generation is deterministic

WOA-09: The system shall allow domain experts to create, edit, or delete entries in the Contextual Activity Matrix.

Acceptance Criteria:

1. Domain experts can modify any cell prior to stage confirmation

WOA-10: The system shall allow domain experts to define Desired and Potential Spans of Action for each Work Task.

Acceptance Criteria:

1. Desired and Potential spans editable

WOA-11: The system shall validate the Contextual Activity Matrix prior to stage confirmation.

Acceptance Criteria:

1. Missing spans flagged
2. Warnings are non-blocking

WOA-12: The system shall allow export of the Contextual Activity Matrix in a text-focused format.

Acceptance Criteria:

1. Export matches UI view

2.4 Stage 3 - Cognitive Transformations Analysis (CTA)

CTA-01: The system shall allow domain experts to create a State–Process Diagram for each Work Task.

Acceptance Criteria:

1. Each Work Task can have zero or more State–Process Diagrams.
2. The system does not force one diagram per task.
3. Domain experts can explicitly mark tasks as not requiring a diagram.

CTA-02: The system shall allow domain experts to create, edit, or delete Cognitive State nodes.

Acceptance Criteria:

1. Nodes and links editable
2. No enforced order

CTA-03: The system shall allow domain experts to create, edit, or delete Cognitive Process links.

Acceptance Criteria:

3. Nodes and links editable
4. No enforced order

CTA-04: The system shall validate State–Process Diagrams for disconnected states or processes.

Acceptance Criteria:

1. The system identifies disconnected states or processes.
2. Validation does not enforce a single canonical ladder structure.

CTA-05: The system shall generate a Decision Ladder template for each Work Task derived from prior-stage artifacts.

Acceptance Criteria:

1. Template provided but editable
2. Templates merge relevant prior artifacts
3. Generation is deterministic

CTA-06: The system shall allow domain experts to edit or remove elements of the Decision Ladder.

Acceptance Criteria:

1. Any element removable

CTA-07: The system shall validate Decision Ladders for orphaned states or processes.

Acceptance Criteria:

1. The system identifies orphaned ladder elements.
2. Validation does not enforce a single canonical ladder structure.

CTA-08: The system shall allow export of State–Process Diagrams and Decision Ladders.

Acceptance Criteria:

1. Diagrams and ladders export correctly

CTA-09: The system shall allow and handle import of outside applications specific to this stage (Qualtrics).

Acceptance Criteria:

1. System allows import of qualtrics projects
2. System takes the project and converts it into the format of our decision ladders
3. Final structure of decision ladder is consistent with the imported version

2.5 Stage 4 - Strategies Analysis (STR)

STR-01: The system shall allow analysts to identify Cognitive Strategies associated with Decision Ladder processes.

Acceptance Criteria:

1. Strategies linked to ladder processes
2. Analysts can add or remove freely

STR-02: The system shall allow analysts to create, edit, or delete Cognitive Strategies.

Acceptance Criteria:

1. Strategies linked to ladder processes
2. Analysts can add or remove freely

STR-03: The system shall allow analysts to document reasoning or conditions associated with each strategy.

Acceptance Criteria:

1. Text fields editable
2. Multiple rationales allowed

STR-04: The system shall generate Strategy Diagrams.

Acceptance Criteria:

1. Diagrams reflect strategies accurately
2. Generation is deterministic

STR-05: The system shall generate an editable, text-focused representation of Cognitive Strategies and reasoning.

Acceptance Criteria:

1. Table auto-generated
2. Editable entries

STR-06: The system shall allow export of Strategy Diagrams and strategy tables.

Acceptance Criteria:

1. Diagram and table export correctly
2. Output is in a spreadsheet format

2.6 Stage 5 - Cognitive Processing Analysis (CPA)

CPA-01: The system shall allow analysts to classify each Cognitive Strategy as Skill-, Rule-, or Knowledge-Based.

Acceptance Criteria:

1. Analysts assigns classification
2. No automatic enforcement

CPA-02: For Skill- and Knowledge-Based strategies, the system shall allow documentation of information use, access, transformation, and application.

Acceptance Criteria:

1. Notes editable and persistent

CPA-03: For Rule-Based strategies, the system shall allow documentation of procedures or activity sequences.

Acceptance Criteria:

1. Procedures editable
2. Multiple steps allowed

CPA-04: The system shall generate Cognitive Processing tables mapping tasks to processing modes.

Acceptance Criteria:

1. Tables map tasks and processing

CPA-05: The system shall allow export of Cognitive Processing tables and updated Strategy Diagrams.

Acceptance Criteria:

1. Tables and diagrams export correctly

2.7 Stage 6 - Social Transactions Analysis (STA)

STA-01: The system shall generate a Social Transactions Docket based on the Contextual Activity Matrix.

Acceptance Criteria:

1. Docket reflects CAM tasks

STA-02: The system shall allow domain experts to assign Agents to Work Tasks.

Acceptance Criteria:

1. Analyst selects agents per task

STA-03: The system shall allow domain experts to specify Transaction Demands and Transaction Modes.

Acceptance Criteria:

1. Fields editable

STA-04: The system shall generate a Transaction Network Diagram.

Acceptance Criteria:

1. Diagram auto-generated from docket

STA-05: The system shall allow domain experts to edit nodes and connections in the Transaction Network Diagram, excluding Work Tasks.

Acceptance Criteria:

1. Nodes/edges editable except tasks

STA-06: The system shall validate the Transaction Network Diagram for orphaned nodes or connections.

Acceptance Criteria:

1. The system identifies orphaned agents, transactions, or connections.
2. The system verifies that all Work Tasks are represented.
3. Validation results are exportable with the diagram.

STA-07: The system shall allow export of Social Transactions Dockets and Transaction Network Diagrams.

Acceptance Criteria:

1. Docket and diagram export correctly

3. Data Management Requirements (DATA)

DATA-01: The system shall store uploaded documents in their entirety.

Acceptance Criteria:

1. All supported input formats are ingested without data loss
2. No data truncation or alteration of source content
3. Stored sources remain accessible to the user

DATA-02: The system shall store all generated and user-confirmed CWA artifacts in a structured artifact repository.

Acceptance Criteria:

1. Artifacts are stored with unique identifiers and explicit stage ownership
2. Artifact storage preserves dependencies and upstream-downstream relationships
3. Automatically generated artifacts are stored distinctly from domain expert confirmed artifacts
4. Artifact storage supports querying and reuse by downstream stages

DATA-03: The system shall support ingestion of structured and unstructured input formats.

Acceptance Criteria:

1. Supported input formats include PDF, DOC, DOCX, TXT, LaTeX, Markdown, CSV, XML, and JSON
2. Identical input files produce identical parsed representations
3. Structured inputs preserve field structure

DATA-04: The system shall support export of confirmed artifacts in text-focused formats suitable for review and sharing.

Acceptance Criteria:

1. Supported formats include PDF, LaTeX, and HTML
2. Exports reflect the current confirmed project state
3. No truncation or loss of artifact content occurs
4. Exports include traceability metadata

4. Usability and Accessibility Requirements

USE-01: The system shall be usable by domain experts without prior knowledge of CWA methods.

Acceptance Criteria:

1. Domain experts can complete a basic end-to-end workflow using in-tool guidance
2. Instructions are framed in domain language rather than CWA terminology
3. Users are not required to understand CWA theory to produce valid artifacts

USE-02: The system shall provide stage-specific instructional content and examples.

Acceptance Criteria:

1. Guidance explains what the user should look for and how to use the tool

2. Novice users receive step-by-step instructions
3. Expert users can suppress instructional content

USE-03: The system shall support controlled, iterative navigation across CWA stages while enforcing stage-freeze rules.

Acceptance Criteria:

1. Users may review artifacts from completed stages at any time
2. Editing a completed stage requires explicit confirmation and downstream artifact discard
3. The system clearly communicates navigation consequences before changes occur.

5. Security and Privacy Requirements (SEC)

SEC-01: The system shall support optional access control for proprietary or sensitive projects.

Acceptance Criteria:

1. Projects can be marked as public or restricted
2. Restricted projects require authentication to access
3. Access control settings are configurable per project

SEC-02: The system shall support optional user authentication for project access and collaboration.

Acceptance Criteria:

1. Users may authenticate to create, access, or share restricted projects
2. Authentication remains optional for public or unrestricted projects
3. Authenticated users can be granted project-specific access levels

SEC-03: The system shall allow anonymization of human subject data where required.

Acceptance Criteria:

1. The system supports removal or masking of identifiers
2. Anonymization applies to stored data and exports
3. Analysts can review anonymized output before export

SEC-04: The system shall provide mechanisms that enable compliance with applicable Institutional Review Board (IRB) requirements for the storage, processing, and export of human-subject data.

Acceptance Criteria:

1. The system provides mechanisms for anonymization, access control, and consent tracking
2. The system does not require storage of identifiable human-subject data
3. Documentation explicitly describes how the system supports IRB-compliant workflows

6. Open Source and Licensing Requirements (OSS)

OSS-01: The system shall be released under an MIT or equivalent permissive open-source license

Acceptance Criteria:

1. License file is present and unambiguous.

OSS-02: the system shall provide complete developer and user documentation

Acceptance Criteria:

1. User documentation is sufficient to install, configure, and operate the system
2. Developer documentation is sufficient to build, modify, and extend the system
3. Example datasets or mock projects are provided without sensitive data.

OSS-03: the system source code shall be publicly available upon project completion

Acceptance Criteria:

1. Source code is publicly accessible in a version-controlled repository.

OSS-04: the system source code shall only use open-source dependencies with licenses compatible with the system's primary license

Acceptance Criteria:

1. All dependencies are released under OSI-approved open source licenses.
2. Dependency licenses are documented and reviewed for compatibility
3. No proprietary or restrictive licensing dependencies are required to build or run the system

7. Deployment Requirements (DEP)

DEP-01: the system shall be accessible through a web-based interface

Acceptance Criteria:

1. System accessible via a browser
2. No client-side installation is required for end users

DEP-02: the system shall be available for free public use

Acceptance Criteria:

1. No paywall or license gate

8. Interpretive Notes and Design Assumptions

Automation Design Principle: Where requirements specify that the system “assists” the domain expert, the specific automation mechanism is intentionally unconstrained. The requirements

specify that the mechanism must be deterministic or configurable, that the outputs must be explainable and traceable, and that domain experts must retain full control over acceptance or rejection. In development, the mechanism might look like a keyword search, pattern matching, rule-based extraction, an LLM with constraints, or manual tagging with suggestions, but it will follow the limits of the requirements.

User Model: The system assumes a single primary domain expert responsible for producing and confirming artifacts within a project. Multi-analyst simultaneous collaboration and role-based workflows are assumed to be out of scope.

Data Volume and Scale: The system shall be designed to support analyses involving dozens of source documents and corresponding artifacts across all of the stages of CWA without loss of responsiveness for typical analyst workflows.

Novice Users: A user with minimal prior experience to CWA is assumed to be knowledgeable about the work domain, but not trained in CWA methodology. The system provides guidance focused on tool use and artifact creation rather than CWA theory.

User Stories

A. Core Project & Data Management Stories

Epic A1 - Project Management and Persistence: As a Domain Expert, I want to create and manage multiple CWA projects so that analyses for different domains remain independent, persistent, and reproducible.

User Story A1.1: As a Domain Expert, I want to create, open, rename, and delete CWA projects so that I can manage analyses across multiple domains and studies.

Traces to: GEN-01

User Story A1.2: As a Domain Expert, I want each project's data and artifacts to persist across sessions so that no analytical work is lost between uses.

Traces to: GEN-01, GEN-02

Epic A2 - Artifact Storage, Traceability, and Provenance: As a Domain Expert, I want all artifacts to be stored with traceability and provenance so that every analytical decision can be reviewed and justified.

User Story A2.1: As a Domain Expert, I want every artifact to reference its originating documents, interviews, or surveys so that I can trace analytical elements back to their source evidence.

Traces to: GEN-03

User Story A2.2: As a Domain Expert, I want to view provenance metadata for each artifact so that I understand how, when, and by whom it was created or modified.

Traces to: GEN-04

User Story A2.3: As a Domain Expert, I want automatically generated artifacts to be clearly labeled as drafts so that I never mistake assisted outputs for authoritative analysis.

Traces to: GEN-07

Epic A3 - User Creation and Management: As a Domain Expert, I want to create and manage an account that holds my projects so that only I can access them from anywhere securely.

User Story A3.1: As a Domain Expert, I want to be able to create an account within the system so that I can access my projects from various devices and secure my data from other users

Traces to: GEN-09

User Story A3.2: As a Domain Expert, I want to be able to log in to my account so that I can access my projects

Traces to: GEN-09

User Story A3.3: As a Domain Expert I want to be able to log out of my account so that I can end my current session and prevent other users of the same device to act as me

Traces to: GEN-09

User Story A3.4: As a Domain Expert, I want to be able to delete my account so that I can scrub my work and documents from the system's storage

Traces to: GEN-09

B. Stage-Specific User Stories

Stage 1 - Work Domain Analysis (WDA)

Epic B1 - Constructing the Abstraction-Decomposition Space: As a Domain Expert, I want to construct and iteratively refine a Work Domain model that faithfully represents domain constraints and purposes.

User Story B1.1: As a Domain Expert, I want to upload and view source documents so that I can ground the Work Domain Analysis in authoritative materials.

Traces to: WDA-01, DATA-01, DATA-02

User Story B1.2: As a Domain Expert, I want the system to suggest candidate Domain Purposes with source references so that I can identify high-level goals efficiently without losing traceability.

Traces to: WDA-02, WDA-03

User Story B1.3: As a Domain Expert, I want to manually create, edit, or delete Domain Purposes so that the abstraction hierarchy reflects my expert judgment.

Traces to: WDA-04

User Story B1.4: As a Domain Expert, I want to build and edit all five abstraction hierarchy levels so that the Work Domain is represented consistently with CWA theory.

Traces to: WDA-05 through WDA-14

User Story B1.5: As a Domain Expert I want to define means–ends links and decomposition levels so that the Abstraction–Decomposition Space can be read as a coherent narrative.

Traces to: WDA-15, WDA-16

User Story B1.6: As a Domain Expert, I want the system to flag missing or inconsistent links so that I can review internal validity without being blocked from proceeding.

Traces to: WDA-17

Stage 2 - Work Organization Analysis (WOA)

Epic B2 - Modeling Work Situations and Tasks: As a Domain Expert, I want to model how work is organized across situations and tasks so that domain functions are contextualized.

User Story B2.1: As a Domain Expert, I want assistance identifying candidate Work Situations so that I can distinguish different operational contexts.

Traces to: WOA-01, WOA-02

User Story B2.2: As a Domain Expert, I want to create and describe Work Tasks so that task definitions are explicit and reviewable.

Traces to: WOA-03, WOA-04

User Story B2.3: As a Domain Expert, I want to map Work Tasks to Domain or Physical Functions when appropriate so that organizational structure aligns with the Work Domain model.

Traces to: WOA-05, WOA-06

User Story B2.4: As a Domain Expert, I want to generate and edit a Contextual Activity Matrix so that task–situation relationships are made explicit.

Traces to: WOA-08, WOA-09

Stage 3 - Cognitive Transformations Analysis (CTA)

Epic B3 - Modeling Cognitive States and Processes: As a Domain Expert, I want to model how work tasks are cognitively executed without enforcing a single canonical process.

User Story B3.1: As a Domain Expert, I want assistance identifying Cognitive States and Processes freely so that narratives can start and progress at any point in the diagram.

Traces to: CTA-02, CTA-03

User Story B3.2: As a Domain Expert, I want editable Decision Ladder templates so that standard representations are available without being prescriptive.

Traces to: CTA-05, CTA-06

Stage 4 - Strategies Analysis (STR)

Epic B4 - Capturing Alternative Cognitive Strategies: As a Domain Expert, I want to document alternative strategies and rationales so that variability in cognitive work is preserved.

User Story B4.1: As a Domain Expert, I want to associate strategies with Decision Ladder processes so that alternative paths through cognitive work are explicit.

Traces to: STR-01, STR-02

User Story B4.2: As a Domain Expert, I want assistance documenting reasoning, conditions, and rules of thumb so that strategy selection can be understood and reviewed.

Traces to: STR-03

Stage 5 - Cognitive Processing Analysis (CPA)

Epic B5 - Identifying Processing Modes: As a Domain Expert, I want to classify strategies by cognitive processing mode so that interface design implications can be derived.

User Story B5.1: As a Domain Expert, I want assistance classifying strategies as skill-, rule-, or knowledge-based so that different cognitive demands are made explicit.

Traces to: CPA-01

User Story B5.2: As a Domain Expert, I want assistance documenting information use and transformation so that cognitive processing can be mapped to interface requirements.

Traces to: CPA-02, CPA-03

Stage 6 - Social Transactions Analysis (STA)

Epic B6 - Modeling Social and Organizational Interactions: As a Domain Expert, I want to model social transactions and authority structures so that cooperative work is represented.

User Story B6.1: As a Domain Expert, I want a Social Transactions Docket generated from existing work tasks so that social analysis builds directly on prior stages.

Traces to: STA-01

User Story B6.2: As a Domain Expert, I want assistance assigning agents, transaction demands, and modes so that responsibility and communication structures are explicit.

Traces to: STA-02, STA-03

C. Safeguards & Meta Stories

Epic C1 - Analyst Trust and Methodological Integrity: As a Domain Expert, I want safeguards that prevent silent errors or over-automation so that the integrity of the analysis is preserved.

User Story C1.1: As a Domain Expert, I want all validation results to appear as blockers so that methodological flexibility is maintained.

Traces to: WDA-17, WOA-11, CTA-04, STA-06

User Story C1.2: As a Domain Expert, I want to navigate linearly across CWA stages so that analysis can proceed iteratively.

Traces to: USE-03

User Story C1.3: As a Domain Expert, I want instructional guidance and examples at each stage so that I can perform analyses correctly even without deep prior expertise.

Traces to: USE-01, USE-02

Work Breakdown Structure

1. Project Management and Process Control

1.1 Agile Scrum Execution

- 1.1.1 Sprint planning and backlog refinement
- 1.1.2 Sprint reviews and retrospectives
- 1.1.3 Stakeholder review meetings

1.2 Product and Backlog Management

- 1.2.1 Product backlog creation and prioritization
- 1.2.2 User story refinement and acceptance criteria maintenance
- 1.2.3 Epic and milestone tracking

1.3 Quality Assurance and Standards Compliance

- 1.3.1 Definition of Done enforcement
- 1.3.2 Test planning and coverage tracking
- 1.3.3 Code review and pull request governance

2. Core System Infrastructure

2.1 Application Architecture

- 2.1.1 Web application framework setup
- 2.1.2 API and service layer design
- 2.1.3 Client–server communication mechanisms

2.2 Project and User Management

- 2.2.1 Project creation, persistence, and deletion
- 2.2.2 Optional user account creation and authentication

2.3 Data Persistence and Schema Design

- 2.3.1 Artifact identity and typing model
- 2.3.2 Stage relationship enforcement
- 2.3.3 Database migration and recovery mechanisms

3. Artifact Governance and Integrity

3.1 Traceability Framework

- 3.1.1 Source document ingestion and indexing
- 3.1.2 Artifact-to-source reference model
- 3.1.3 Navigation from artifact to source

3.2 Provenance and Metadata Management

- 3.2.1 Provenance schema definition

- 3.2.2 Creation and modification history tracking
 - 3.2.3 Draft and analyst-confirmed state handling
- 3.3 Change Impact and Propagation Control
 - 3.3.1 Analyst reconciliation workflows
 - 3.3.2 Propagation decision logging
- 4. Manual Artifact Creation and Editing**
 - 4.1 Stage 1 – Work Domain Analysis
 - 4.1.1 Domain purpose creation and editing
 - 4.1.2 Domain value creation and editing
 - 4.1.3 Domain function creation and editing
 - 4.1.4 Physical function creation and editing
 - 4.1.5 Physical object creation and editing
 - 4.1.6 Means–ends and decomposition link editing
 - 4.2 Stage 2 – Work Organization Analysis
 - 4.2.1 Work task creation and editing
 - 4.2.2 Task–function association management
 - 4.2.3 Contextual Activity Matrix editing
 - 4.3 Stage 3 – Cognitive Transformations Analysis
 - 4.3.1 State–process diagram element editing
 - 4.3.2 Decision ladder element editing
 - 4.4 Stage 4 – Strategies Analysis
 - 4.4.1 Cognitive strategy creation and editing
 - 4.4.2 Strategy–process association management
 - 4.4.3 Strategy rationale documentation
 - 4.5 Stage 5 – Cognitive Processing Analysis
 - 4.5.1 Processing mode classification
 - 4.5.2 Information use and procedure documentation
 - 4.6 Stage 6 – Social Transactions Analysis
 - 4.6.1 Agent creation and editing
 - 4.6.2 Transaction demand and mode editing
 - 4.6.3 Task–agent association

5. Visualization and Navigation

5.1 Interactive Modeling Environments

- 5.1.1 Abstraction Hierarchy Editor
- 5.1.2 Abstraction-Decomposition Space Mapper
- 5.1.3 Contextual Activity Matrix Editor
- 5.1.4 State-Process Diagram Editor
- 5.1.5 Decision Ladder Visualizer
- 5.1.6 Strategy Diagram Mapper
- 5.1.7 Social Transaction Docket Editor
- 5.1.8 Transaction Network Diagram Mapper

5.2 Supplemental Information Displays

- 5.2.1 Non-diagrammatic data views
- 5.2.2 Metadata and node property inspector

5.3 Automated Model Validation

- 5.3.1 Orphaned Node detection
- 5.3.2 Connectivity validation
- 5.3.3 Cross-stage mapping validation
- 5.3.4 Conflict and Inconsistency warning system

5.4 Workflow Cycle Support

- 5.4.1 Linear Stage Progression
- 5.4.2 Breadcrumb navigation
- 5.4.3 Impact Flag visualization
- 5.4.4 Analyst Resolution log

6. Assisted and Automated Features

6.1 Deterministic Assistance Mechanisms

- 6.1.1 Keyword and pattern-based extraction
- 6.1.2 Rule-based candidate identification
- 6.1.3 Configurable assistance parameters

6.2 Stage-Specific Assistance

- 6.2.1 Work Domain Analysis candidate generation
- 6.2.2 Work Organization Analysis candidate suggestions
- 6.2.3 Cognitive Analysis link candidate suggestions
- 6.2.3 Template-based assistance for later stages

6.3 Explainability and Control

- 6.3.1 Assistance output explanation mechanisms

- 6.3.2 Analyst acceptance and rejection controls
- 6.3.3 Determinism and reproducibility testing
- 6.3.4 System guidance on CWA Process

7. Export, Interoperability, and Reporting

- 7.1 Export Infrastructure
 - 7.1.1 PDF export
 - 7.1.2 LaTeX export
 - 7.1.3 HTML export
 - 7.1.4 CSV export
- 7.2 Provenance and Traceability Preservation
 - 7.2.1 Metadata embedding in exports
 - 7.2.2 Traceability rendering in exported artifacts

8. Usability, Guidance, and Onboarding

- 8.1 Instructional Content
 - 8.1.1 Stage-specific guidance text
 - 8.1.2 Worked examples and mock projects
 - 8.1.3 CWA Process walkthrough on first project
 - 8.1.4 Sample output artifacts
- 8.2 Novice Analyst Support
 - 8.2.1 Context-sensitive help
 - 8.2.2 Non-intrusive user interface affordances

9. Security, Privacy, and Compliance

- 9.1 Access Control
 - 9.1.1 Authentication enforcement
- 9.2 Human Subject Data Protection
 - 9.2.1 Anonymization and masking tools
 - 9.2.2 Consent tracking mechanisms
- 9.3 IRB Compliance Support
 - 9.3.1 Documentation of compliant workflows
 - 9.3.2 Export controls for sensitive data

10. Open-Source Release and Deployment

- 10.1 Open-Source Readiness
 - 10.1.1 License inclusion, dependencies

10.1.2 Repository organization

10.1.3 Contribution guidelines

10.2 CI/CD and Quality Gates

10.2.1 Linting and test automation

10.2.2 Test coverage enforcement

10.2.3 Containerized deployment verification

10.3 Documentation and Community Support

10.3.1 Developer documentation

10.3.2 User documentation

Communication and Stakeholder Management Plan

This plan defines how communication and stakeholder engagement will be managed throughout the development of the Cognitive Work Analysis Support Tool (CWAST). Using Agile Scrum methodology, the project emphasizes frequent communication, transparency, and iterative feedback to ensure technical quality, methodological integrity, and stakeholder alignment.

Stakeholder Identification and Roles

Stakeholder	Role/Interest	Level of Influence	Engagement
Product Owner - Dr. Esa Rantanen	Defines product goals, prioritizes backlog, ensures CWA fidelity	High	Weekly sprint reviews, backlog refinement
Scrum Master - Ella Natter	Facilitates Scrum process, removes impediments, ensures communication	High	Daily coordination, Scrum event facilitation
Developers	Design, implement, test, and document system increments	High	Daily standups, sprint planning, retrospectives
Coach - J. Scott Hawker	Advises on software engineering practices, architecture, and process	High	Periodic technical reviews, sprint check-ins

Recurring Scrum Events

Event	Participants	Frequency	Purpose	Outputs
Sprint Planning	Product Owner, Scrum Master, Developers	Once per sprint	Define sprint goal and scope	Sprint backlog
Daily Standup	Developers, scrum master	Daily on active work days	Synchronize work, figure out blockers	Standup notes
Sprint Review	Product owner, developers	End of sprint	Demo Increment, collect feedback	accepted/revised backlog items

Sprint Retrospective	Developers, scrum master	End of sprint	Improve processes and collaboration	Action items
----------------------	--------------------------	---------------	-------------------------------------	--------------

Stakeholder Specific Communication

Product Owner: Dr. Esa Rantanen

Channels: MyCourses, Email, Sprint Reviews, Discord

Frequency: Weekly

Focus: feature acceptance, backlog prioritization, validation of CWA methodological assumptions

Software Development Coach: J. Scott Hawker

Channels: MyCourses, Email, Sprint Reviews, Technical Design Discussions

Frequency: Periodic

Focus: Code quality and architecture, Agile and Scrum practice adherence, tooling, CI/CD

Feedback and Decision Management: feedback from all stakeholders is captured as Product Backlog Items. The Product Owner resolves priority or scope conflicts. Technical process recommendations from the coach are discussed during sprint planning and retrospectives. Feature acceptance is determined during the Sprint Review, based on the acceptance criteria.

Documentation and Traceability: decisions and rationale are recorded on Confluence. Jira tickets trace requirements to implementation. GitLab pull requests link code changes to backlog items. This supports traceability and transparency in the communication plan.

Review and Adaptation: this plan will be reviewed during retrospectives and updated as stakeholder needs evolve.

Communication Reasoning: Mycourses is intended to be the main repository for documentation due to its well defined structure. This is where long term communication such as Sprint Retrospectives and Backlog Refinements will be stored. Discord will be used to asynchronously ask miscellaneous domain questions informally to the Product Owner. Email is the primary form of communication with the Coach, including meeting agendas, notes, and notifications of document addition to Mycourses.

Risk Management Plan

The purpose of this Risk Management Plan is to identify, assess, and manage risks that could impact the successful delivery, quality, or integrity of the Cognitive Work Analysis Software Tool (CWAST). Risk management is treated as a continuous activity, with risks reviewed and mitigated throughout development rather than addressed only upfront.

1. Technical Risks

RISK-T1 : Automation mechanisms produce non-deterministic results

Description:

assisted artifact generation may behave inconsistently

Impact: HIGH

Likelihood: LOW

Mitigation:

enforce determinism/configuration, document assumptions, allow for disabling automation

RISK-T2: Assisted artifact generation produces low-quality or noisy suggestions

Description:

Automatically suggested artifacts may be irrelevant, incomplete, or misleading.

Impact: HIGH

Likelihood: MEDIUM

Mitigation:

Treat all assisted outputs as editable drafts, require source traceability, support fully manual workflows, and iteratively tune assistance mechanisms.

RISK-T3: Performance degradation with moderate data volume

Description:

System responsiveness may degrade when handling dozens of documents and artifacts.

Impact: MEDIUM

Likelihood: MEDIUM

Mitigation:

Conduct early performance testing, use lazy loading and incremental rendering, and profile bottlenecks throughout development.

RISK-T4: Visualization complexity exceeds implementation capacity*Description:*

Text visualizations of complex CWA artifacts may be difficult to implement correctly and efficiently.

Impact: MEDIUM

Likelihood: MEDIUM

Mitigation:

Start with minimal viable visualizations, iterate incrementally, and leverage existing literature on CWA artifacts.

RISK-T5: Data volume exceed storage space*Description:*

The amount of documents provided by each analyst during the early stages of CWA that need to be stored for later use might quickly exceed the data stores of this application.

Impact: MEDIUM

Likelihood: MEDIUM

Mitigation:

set limits on how much information the users of the application can expect to be saved. Save CWA artifacts, but not necessarily entire documents provided by the user.

2. Architectural Risks

RISK-A1: Data model rigidity limits evolution*Description:*

Early schema decisions may not accommodate later understanding of CWA artifacts.

Impact: HIGH

Likelihood: MEDIUM

Mitigation:

iterative refinement of the database driven by sprint feedback.

RISK-A2: Frontend–backend format instability*Description:*

Frequent API changes may slow development and introduce defects.

Impact: MEDIUM

Likelihood: MEDIUM

Mitigation:

Define stable API contracts early, use contract tests, and clearly assign ownership for interface changes.

RISK-A3: Premature architectural decisions constrain later stages

Description:

Early design choices may limit flexibility needed for later CWA stages.

Impact: MEDIUM

Likelihood: MEDIUM

Mitigation:

Defer irreversible decisions, validate architecture through working increments, and revisit design during retrospectives.

3. Process and Schedule Risks

RISK-P1: Scope creep due to open-ended research goals

Description:

The project scope may expand beyond available time and resources.

Impact: HIGH

Likelihood: MEDIUM

Mitigation:

Maintain strict backlog prioritization, define sprint goals clearly, and defer non-core features.

RISK-P2: Uneven stakeholder availability across sprints

Description:

Academic schedules or workload imbalance may reduce team capacity.

Impact: MEDIUM

Likelihood: MEDIUM

Mitigation:

Use small, independent stories, flexible sprint scope, and shared ownership of components.

RISK-P3: Scrum practices are inconsistently applied

Description:

Standups, reviews, or retrospectives may be skipped or de-emphasized.

Impact: MEDIUM

Likelihood: LOW

Mitigation:

Scrum Master enforces ceremonies, keeps processes lightweight, and adapts practices during retrospectives.

4. Quality & Maintenance Risks

RISK-Q1: Accumulation of technical debt

Description:

Rapid iteration may lead to shortcuts that reduce maintainability.

Impact: HIGH

Likelihood: MEDIUM

Mitigation:

Enforce CI checks, require code reviews, schedule refactoring stories, and involve the development coach in reviews.

RISK-Q2: Insufficient test coverage

Description:

New features may be added without adequate testing.

Impact: HIGH

Likelihood: LOW–MEDIUM

Mitigation:

Enforce 90% test coverage, require testing stories for all backlog items, and verify tests via peer review.

RISK-Q3: Documentation lags behind implementation

Description:

User and developer documentation may become outdated.

Impact: MEDIUM

Likelihood: MEDIUM

Mitigation:

Require Confluence updates after merges and treat documentation as part of the Definition of Done.

Development Methodology

The development of this project will be done using an Agile Scrum Methodology. Agile is an iterative approach focusing on flexibility, customer collaboration, and small functional releases. The scope and requirements of this project are not well defined, so Agile should allow for changing requirements. The sprint-and-release workflow lends itself well to weekly meetings with the project sponsor. Agile Scrum is also most familiar to the development team.

Planned Activities and Artifacts

The scrum process that we will use has four key activities or events.

1. *Sprint Planning*: The team will define the work to be performed during the upcoming sprint. This will occur at the same meeting as the previous sprint's retrospective.
2. *Daily Standup*: a daily report by each team member describing what they worked on, and what they are currently working on. For our specific project, developers are only expected to respond to the standup on days that we complete work.
3. *Sprint Review*: held at the end of the sprint to inspect and edit the product backlog
4. *Sprint Retrospective*: a retrospective on team progress and communication. A plan is created for improvements to be enacted during the next sprint.

Three key artifacts are involved in the Scrum methodology. These artifacts will be maintained throughout the entirety of the project.

1. *Product Backlog*: a prioritized list of all features, requirements, and fixes for the product
2. *Sprint Backlog*: a subset of items selected to be completed for the current sprint.
3. *Increment*: the sum of all completed work that meets quality standards at the end of the sprint.

Roles

Product Owner: The product owner is responsible for defining the product goal, prioritizing work in the Product backlog, representing stakeholders, and ensuring transparency. The product owner for this project is Dr. Esa Rantanen

Scrum Master: The scrum master is accountable for ensuring that scrum practices are understood and enacted. Their responsibilities include facilitating scrum events, removing impediments, and finding ways to improve the project. The scrum master for this project is Ella Natter.

Developers: Developers are accountable for creating a usable Increment during each sprint. Their responsibilities include designing, building, testing, and delivering the product increment,

and organizing to achieve the goals of the sprint. The developers on this project are Ella Natter, Ash Franklin, Ata Noor, Cole DenBleyker, Jensen Derosier, and Zach Brown.

Stakeholders: Stakeholders are individuals or groups who can provide feedback, insights, and requirements that help guide future iterative development. The key stakeholders for this project are the RIT Psychology Department, Dr. Esa Rantanen, J. Scott Hawker, and the open-source community at RIT.

Standards and Quality Practices

All stories that are in the Sprint Backlog must have an associated testing story. Developers are responsible for writing tests for the features that they add, but another developer must verify the tests. All written tests must cover the happy path, failing conditions, and edge cases. Every method that is written must also have proper documentation in the form of docstrings. Upon pushing code to the repository, a GitHub CI must run through the code linter, the unit test suite, and a Docker image run, and pass all of those checks. Every merge must have a pull request approved by two other developers. Merged pull requests must be followed by updating confluence documentation relevant to the changes. The code must achieve 90% test coverage as determined by SonarQube.

Tools

Jira will help facilitate proper Scrum methodology and keep track of various performance and/or progressive metrics. Jira will also allow us to keep track of each of our own tasks and hold each other accountable if someone is in the red for their work. GitHub PRs will be used in tandem to keep track of features and encourage discussion on the individual PR for anything related to that specific implementation. Confluence will be used to hold and organize past artifacts.

Schedule

As is consistent with the Agile Scrum Methodology, the Product Owner (aka sponsor) should expect a full working Increment at the end of each sprint. The Increment should include full completed features from the prioritized sprint backlog.

Process & Project Metrics

Process

This project will adhere to the Software Development Life Cycle (SDLC). The SDLC includes seven steps— Planning, Analysis, Design, Development/Coding, Testing, Deployment, and Maintenance.

- a. Planning: define goals, scope, and resources
- b. Analysis: study requirements, feasibility, and necessary resources
- c. Design: Create the system's blueprint, architecture, and user interface
- d. Development/Coding: write the actual code based on the design
- e. Testing: verify quality, performance, and find bugs
- f. Deployment: release the software to users
- g. Maintenance: ongoing support, updates, and improvements

While the project adheres to an Agile Scrum development methodology, the SDLC phases are used as a conceptual framework for categorizing and analyzing work activities rather than as a sequential lifecycle. In practice, planning, analysis, design, implementation, testing, and documentation occur iteratively within each sprint, consistent with Agile principles.

Metrics

Chosen Progress Metric:

Earned Value Chart: identify the activities for your project, and associate an earned value (# of points) with each. You earn the value (only) when you complete the activity - no points for partially completed activities. Plot total points earned against time.

Earned Value will be tracked at the level of completed backlog items (user stories or tasks).

Value is earned only when a backlog item meets its Definition of Done. This metric is used to visualize delivery trends over time rather than to enforce fixed-scope predictions.

Chosen Product Metrics:

Effort By Type of Activity: classify effort based on activity type (requirements, design, implementation, coordination, documentation, customer interaction, etc.). Useful to see where your time is going.

Requirements Volatility: percent of requirements that were changed during each week. Used to ensure that analytical rigor and traceability requirements are adequately validated.

Test Cases Per Requirements: the number of test cases per requirements (user stories, use cases). Used to ensure that analytical rigor and traceability requirements are adequately validated.

Average Response Time: performance of specific actions

Tools & Techniques

Summary

Category	Usage	Tool
Project Management Tools	Process	Jira / Confluence
Technical Development Tools	Frontend	Selenium, GitHub
	Backend	Jest, Pytest, SonarQube, Ruff
Technical Implementation Tools	Database	MongoDB
	Backend	Flask
	Frontend	React / NextJs
	Deployment	Nginx and Docker

Project Management Tools

Jira: Jira will be used in this project to plan, track, and manage work. The project will be broken down into tickets representing features or bugs, and those tickets will be assigned to a developer to complete. The status (To Do, In Progress, Done) of the tickets will be visible at all times.

Confluence: Confluence will be used to handle project documentation. Articles can be directly linked to Jira tickets to easily see the relevant documentation. It allows for collaborative editing and will be used as a wiki.

Technical Development Tools

GitHub: GitHub will be used to store the code and check whenever the code is changed. It will be used to automatically run tests, check code quality, build the application, and prepare it for deployment.

Selenium: Selenium is a front end testing tool that can automate user interface testing. It simulated how a real user interacts with the website (e.g., clicking a button), and ensures an app works correctly from a user perspective.

Jest: Jest will be used to test individual pieces of JavaScript code to ensure expected behavior. The tests will run automatically and verify correct functionality.

Pytest: Pytest will be used to test individual pieces of Python code to ensure expected behavior. The tests will run automatically and verify correct functionality.

SonarQube: SonarQube is a tool that scans the codebase, looking for bugs, security risks, performance issues, and maintainability problems.

Ruff: Ruff is a Python linter that checks code for mistakes, inconsistencies, and bad practices. It will be used to catch errors before reaching production, and ensure consistency in code between developers.

Technical Implementation Tools

MongoDB: MongoDB is a document-based database, meaning that data will be stored in flexible structures instead of rigid tables. This will help us manage the unstructured data we are anticipating working with. It should be able to handle a large and complex database.

Flask: Flask is a Python web framework that handles incoming requests from users or other systems, routing URLs to backend logic, and returning responses. It will not force database or structure choices.

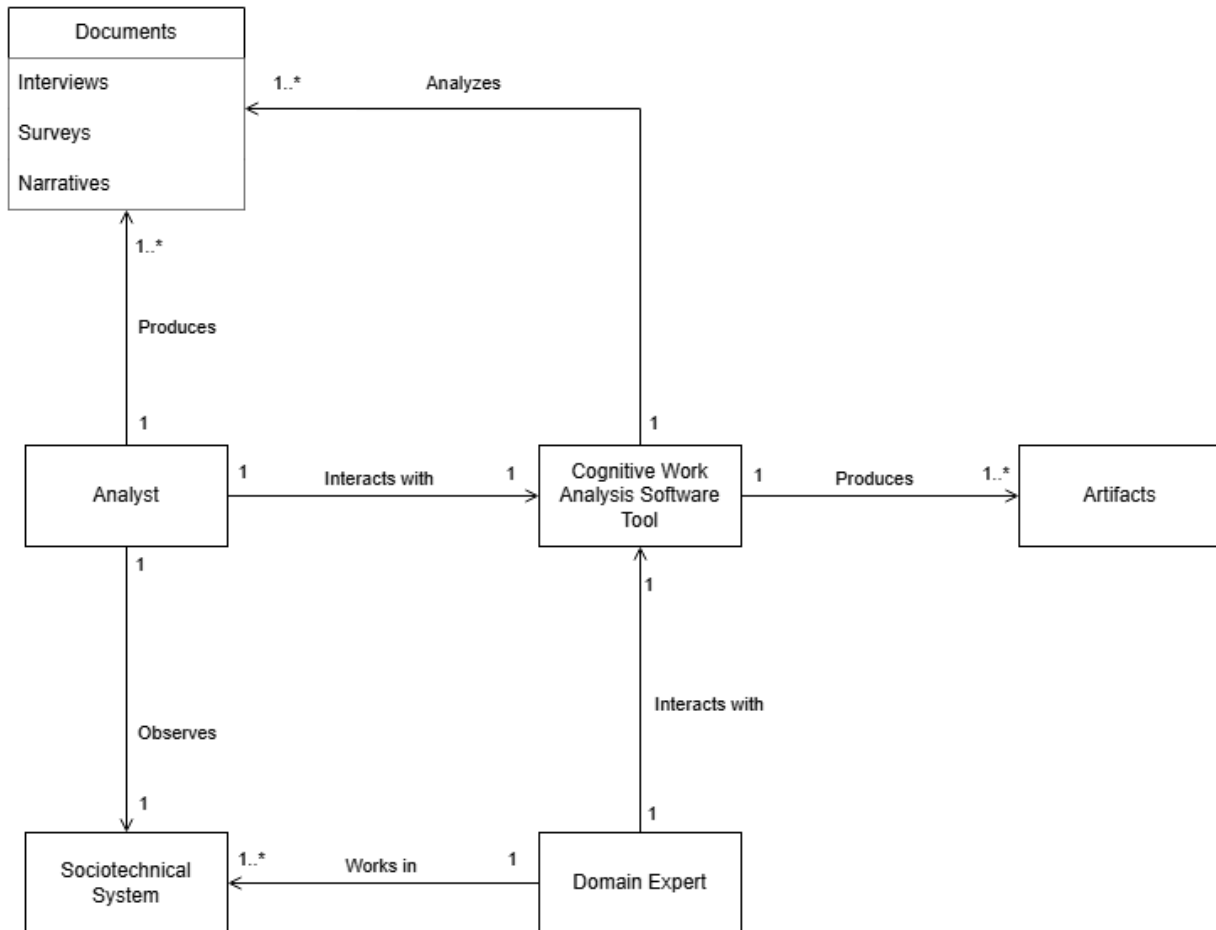
React: React is a framework for building the interactive parts of a web application. Allows dynamically updating UI without a page reload.

Next.js: Next.js is built on top of React and adds server-side rendering (faster initial page loads), Backend API routes, authentication, and session handling. It controls how and where React runs.

NGINX: NGINX sits in front of the application and accepts incoming web traffic. It forwards requests to the backend, handles security, caches responses, and load-balances traffic. It is extremely fast and memory efficient, improves performance, and reduces the load on backend services.

Docker: Docker is used to create environments with predetermined variables so that it will always be set up in a specific way to ensure that the environment does not change. It will be used for deployment.

Domain Model



The Cognitive Work Analysis Software Tool is operated by a Domain Expert who analyzes their Sociotechnical System. The Analyst collects and uploads Documents, including interviews, surveys, and narrative materials derived from the system. From these Documents, the Cognitive Work Analysis Software Tool supports the creation of CWA Artifacts across all six stages. Each Artifact maintains traceability to its source documents and to upstream Artifacts.